INTERVIEW EDITION / 2026 · REV 2

JavaScript Interview Field Guide

Language mechanics, browser runtime, async control, and implementation prompts

WHAT THIS IS FOR

Frontend/full-stack coding screens where fluency matters: explain language behavior, avoid runtime traps, and implement common utilities without library help.

INTERVIEW RULE

A cheatsheet should reduce recall time. It should not replace explaining the invariant, tradeoff, and failure mode in your own words.

HOW TO USE IT

- Read pages 2–3 before a practice block.
- Middle pages: only after identifying the category.
- Final checklist: during timed mocks.
- Re-code the templates from memory; passive rereading is not preparation.

02 Build the correct JavaScript mental model

A. VALUES, IDENTITY, AND EQUALITY

- Primitives copy by value: string, number, bigint, boolean, symbol, undefined, null. Objects/functions copy **references**.
- `===` avoids coercion but `NaN !== NaN` and `0 === -0`. `Object.is` flips both edge cases.
- Map/Set use **SameValueZero**: NaN equals NaN; 0 equals −0; objects still compare by identity.
- Shallow copy (`{...o}`, `[...a]`) shares nested references. `structuredClone` handles cycles, Map, Set, Date; not functions/DOM nodes.

B. COERCION TRAPS

EXPRESSION	RESULT / RULE
`1 + '2'`	`'12'`: + concatenates with a string
`'3' - 1`	`2`: numeric operators coerce
`Boolean([])`	`true`: objects are truthy
`Number('')`	`0`
`typeof null`	`'object'`: historical
`x == null`	true only for null/undefined — the deliberate nullish check

- Falsy complete list: `false, 0, -0, 0n, '', null, undefined, NaN`. Prefer explicit domain checks over truthiness when 0 or empty string is valid.

C. SCOPE, HOISTING, CLOSURES

- `let/const` are block scoped and sit in a temporal dead zone before declaration. `var` is function scoped, initialized to undefined.
- Function declarations are hoisted with bodies. Function expressions follow their variable's rules.
- Closure = function plus access to **lexical bindings, not frozen values**. Each call creates a new environment.

```
function counter() {
  let n = 0;
  return () => ++n;
}
for (let i = 0; i < 3; i++)
  setTimeout(() => console.log(i)); // 0 1 2 (let: per-iteration binding)
```

D. THIS: DECIDED BY THE CALL SITE

CALL	THIS
`obj.f()`	obj
`f()`	undefined in strict/module code
`f.call(x,a)` / `apply(x,[a])`	x, invoked now
`f.bind(x)`	new function bound to x
arrow	lexical; call/bind cannot change

- A detached method loses its receiver: `const m = obj.f; m()`. Event/listener callbacks and timers are common places to notice this.

E. PROTOTYPES AND CLASSES

- A property read checks own properties, then follows the prototype chain. `Object.hasOwn(o,k)` tests only own properties.
- `class` is prototype-based syntax. Instance methods live on `C.prototype`; field arrow functions are per-instance.
- `new C()`: create object → set prototype → call C with this → return the explicit object result if supplied.
- Prefer composition unless inheritance models a stable is-a relationship. Private `#fields` are enforced by the language.

> **SAY THIS**
> "I will state whether the prompt cares about identity, shallow structure, or deep structure before I choose equality or cloning."

03 Collections, iteration, and data transformation

A. WHICH COLLECTION?

NEED	USE	KEY FACT
Ordered indexed data	Array	length; numeric indices
String/symbol property record	Object	prototype + enumeration rules
Any-key dictionary	Map	**identity keys; insertion order**
Unique membership	Set	has/add/delete; insertion order

- Map: set/get/has/delete/clear/size. Set: add/has/delete/clear/size. Arrays use includes/push/splice/length.

B. ARRAY METHODS BY INTENT

INTENT	METHOD	TRAP
Transform	map / flatMap	returns new array
Keep	filter	predicate truthiness
Find	find / findIndex	undefined vs −1
Aggregate	reduce	**always pass initial value**
Test	some / every	short-circuits
Copy range	slice	end exclusive
Edit in place	splice	returns removed items
Order	sort	**mutates; default is string order**

- Non-mutating twins: `toSorted, toReversed, toSpliced, with`. Confirm the runtime, or use spread + the classic method.

C. OBJECTS, ENTRIES, ENUMERATION

- `Object.keys/values/entries` return own enumerable string-keyed properties; symbols need `getOwnPropertySymbols`.
- `Object.fromEntries` rebuilds an object from pairs. `Object.assign(target, ...sources)` mutates target and is shallow.
- `for...in` includes enumerable **inherited** keys; use `Object.entries` for records and `for...of` for iterables.

```
const out = Object.fromEntries(
  Object.entries(obj)
    .filter(([, v]) => v > 0)
    .map(([k, v]) => [k, v * 2])
);
```

D. STRINGS, UNICODE, NUMBERS

- Strings are immutable. `slice` supports negatives; split/join; replaceAll; `localeCompare` for user-facing sort.
- String length counts **UTF-16 code units**, not user-perceived characters. `[...s]` iterates code points, but grapheme clusters may still span several.
- Numbers are IEEE-754 doubles. Integers are exact only through `Number.MAX_SAFE_INTEGER`; use BigInt when required.
- Floating comparison uses domain tolerance, not blindly `Number.EPSILON`. `toFixed` returns a string.

E. COMMON ONE-LINERS

```
const freq = a.reduce((m, x) =>
  m.set(x, (m.get(x) ?? 0) + 1), new Map());

const deduped = [...new Set(a)];

const chunks = Array.from(
  {length: Math.ceil(a.length / n)},
  (_, i) => a.slice(i * n, i * n + n));

const range = Array.from({length: n}, (_, i) => i);

// 2-D init — fill(Array(n)) aliases ONE row:
const grid = Array.from({length: m},
  () => Array(n).fill(0));
```

EDGE CASE PASS

Empty input, sparse arrays, duplicate keys, NaN, −0, mutation, nested references, Unicode, and numeric overflow are the collection questions interviewers use to separate memorization from fluency.

04 Async runtime: event loop, promises, and control

A. EVENT LOOP ORDER

```
console.log('A');
setTimeout(() => console.log('D'));
Promise.resolve().then(() => console.log('C'));
console.log('B');            // A B C D
```

- Run the current job to completion → **drain microtasks** → render opportunity → next task. Promise handlers, `queueMicrotask`, and await continuations are microtasks.
- Timers provide a **minimum delay**, not a schedule guarantee. A long synchronous task blocks input, paint, timers, and promise progress.
- `await` always yields; code after it resumes in a microtask even for a non-promise.

B. PROMISE CONTRACT

- A promise settles **once**. A throw in an executor/handler becomes rejection. Returning a promise from `then` is flattened.
- `catch(f)` is `then(undefined, f)`. `finally` normally passes through the original outcome.

COMBINATOR	FULFILLS	REJECTS
all	all values, input order	first rejection
allSettled	all outcome records	never
race	first settlement	if first settlement rejects
any	first fulfillment	AggregateError if all reject

C. SEQUENTIAL, PARALLEL, LIMITED

```
// sequential
for (const x of xs) out.push(await f(x));

// unbounded parallel
const out = await Promise.all(xs.map(f));

// production: concurrency limit when f hits
// a scarce resource — see §08.C
```

- Choose from dependency capacity and ordering requirements. Parallel is not automatically better; unbounded fanout can trigger rate limits or memory pressure.

D. CANCELLATION AND TIMEOUTS

```
const ctrl = new AbortController();
const signal = AbortSignal.any([
  ctrl.signal, AbortSignal.timeout(5000)
]);
const res = await fetch(url, { signal });
// ctrl.abort('user cancelled')
```

- Cancellation is **cooperative**. Pass the signal through every layer; clean up listeners/work and distinguish abort from failure.
- A promise itself is not cancellable; the underlying operation must observe the signal.
- `AbortSignal.any/timeout` are newer — in older runtimes, compose manually with a timer that calls `ctrl.abort()`.

E. RETRY RULES

- Retry only transient failures and only when the operation is **safe/idempotent**. Bound attempts and total deadline.
- Exponential backoff + jitter; honor Retry-After. Do not stack retries at multiple layers.
- Keep error cause/context. Never swallow rejection with an empty catch. (Implementation: §08.D.)

F. ASYNC ITERATION

- `for await...of` consumes async iterables/streams with backpressure-friendly sequencing.
- `forEach(async ...)` does **not await** callbacks. Use `for...of` or `Promise.all`.
- Streaming parsers must preserve partial frames across chunks and decode with stream mode.

> **SAY THIS**
> "I will define ordering, maximum concurrency, cancellation, timeout, and partial-failure semantics before I write the async loop."

05 Browser runtime and DOM interview essentials

A. DOM EVENT FLOW

- Event path: **capture from root → target → bubble back**. Most listeners use the bubble phase.
- `preventDefault()` cancels the browser action; `stopPropagation()` stops travel; they are different.
- **Delegation:** attach one ancestor listener, locate `event.target.closest(selector)`, and verify it belongs to the container.

```
list.addEventListener('click', e => {
  const btn = e.target.closest('[data-remove]');
  if (!btn || !list.contains(btn)) return;
  remove(btn.dataset.remove);
});
```

B. LISTENER LIFECYCLE

- Remove with the **same function reference** and capture option — or register with `{signal}` and abort.
- Use passive listeners for scroll/touch only when you will not call preventDefault.
- Avoid layout thrash by grouping DOM **reads then writes**; use `requestAnimationFrame` for visual updates.

C. FETCH CORRECTLY

- Fetch rejects on **network/abort, not HTTP 4xx/5xx**. Check `response.ok`. The body can normally be consumed once.
- Encode query parameters; set Content-Type only when appropriate; understand credentials/CORS rather than disabling security.
- Abort stale requests on unmount/new query. Treat timeout, user cancel, offline, server error, and parse error separately.

```
const res = await fetch(url, {signal});
if (!res.ok) throw new Error(`HTTP ${res.status}`);
const data = await res.json();
```

D. STORAGE AND SECURITY

STORE	LIFETIME / SCOPE	WATCH
localStorage	origin, persistent	sync; string only; XSS readable
sessionStorage	tab session	string only
IndexedDB	origin, async	transactions/schema
Cookie	sent by policy	size; CSRF; flags

- Avoid rendering untrusted HTML. Prefer `textContent`/React escaping; sanitize if HTML is a product requirement.
- Do not store long-lived secrets where injected script can read them. Security properties depend on the whole auth design.

E. PERFORMANCE CHECK

- **Measure before optimizing:** input responsiveness, long tasks, layout/paint, network waterfall, memory retention.
- Debounce bursty final work; throttle continuous work; virtualize long lists; lazy-load code/media; cache with invalidation.
- Workers move CPU work off the main thread, but data transfer/serialization and cancellation still matter.

> **WIDGET DEFINITION OF DONE**
> Keyboard works, focus is visible, loading/empty/error states exist, stale requests are canceled, listeners are cleaned up, and the UI survives rapid repeated input.

06 Implementation prompts I: rate control

A. DEFINE THE CONTRACT FIRST

- Leading/trailing? Preserve `this` and arguments? Return value? Cancel/flush? Promise-aware? Error behavior?
- State the minimal contract you will implement, then mention extensions. This prevents solving the wrong debounce or curry.

B. DEBOUNCE

```
function debounce(fn, ms) {
  let timer;
  function wrapped(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), ms);
  }
  wrapped.cancel = () => clearTimeout(timer);
  return wrapped;
}
```

- Trailing edge: every call resets the timer. Test rapid calls, preserved receiver/args, cancel, and a later independent burst.
- Common follow-up — `flush()`: store the pending args/ receiver; flush clears the timer and invokes immediately.

C. THROTTLE (LEADING, THEN WITH TRAILING)

```
function throttle(fn, ms) {       // leading-only
  let ready = true;
  return function(...args) {
    if (!ready) return;
    ready = false;
    fn.apply(this, args);
    setTimeout(() => { ready = true; }, ms);
  };
}

function throttleT(fn, ms) {      // + trailing call
  let ready = true, pending = null;
  return function tick(...args) {
    if (!ready) { pending = [this, args]; return; }
    ready = false;
    fn.apply(this, args);
    setTimeout(() => {
      ready = true;
      if (pending) {
        const [t, a] = pending; pending = null;
        tick.apply(t, a);         // latest call wins
      }
    }, ms);
  };
}
```

07 Implementation prompts II: caching and composition

A. ONCE AND MEMOIZE

```
const once = fn => {
  let done = false, value;
  return function(...args) {
    if (!done) { value = fn.apply(this, args); done =
true; }
    return value;
  };
};
```

- Caveat: if fn **throws**, `done` is still false here, so the next call retries. Set `done = true` first if "attempted once" is the contract — say which you chose.

```
function memoize(fn, key = (...a) => JSON.stringify(a)) {
  const cache = new Map();
  return function(...args) {
    const k = key(...args);
    if (!cache.has(k))
      cache.set(k, fn.apply(this, args));
    return cache.get(k);
  };
}
```

- Discuss thrown errors and rejected promises: cache them or allow retry? JSON keys fail on cycles, key-order differences, functions, and identity-sensitive objects.

B. CURRY / PARTIAL APPLICATION

```
const curry = fn => function curried(...a) {
  return a.length >= fn.length
    ? fn.apply(this, a)
    : function(...b) {
        return curried.apply(this, [...a, ...b]);
      };
};
```

- `fn.length` ignores rest parameters and stops at the first default; an explicit arity is more robust.
- Clarify whether multiple arguments per call are allowed and whether extra arguments pass through.

C. COMPOSE / PIPE / CLASSNAMES

```
const pipe = (...fns) => x =>
  fns.reduce((v, f) => f(v), x);

// classNames with the full canonical contract:
// strings/numbers kept, arrays recursed,
// objects contribute keys with truthy values
function classNames(...xs) {
  const out = [];
  for (const x of xs) {
    if (!x) continue;                  // falsy dropped
    if (typeof x === 'string' || typeof x === 'number')
      out.push(x);
    else if (Array.isArray(x))
      out.push(classNames(...x));      // recurse
    else if (typeof x === 'object')
      for (const k in x) if (x[k]) out.push(k);
  }
  return out.filter(Boolean).join(' ');
}
```

- Clarify dedupe, later-value-wins for objects, and whether nested arrays appear — then scope aloud.

08 Implementation prompts III: async control

A. PROMISE.ALL SKELETON

```
function promiseAll(items) {
  const xs = [...items];
  return new Promise((resolve, reject) => {
    if (!xs.length) return resolve([]);
    const out = new Array(xs.length);
    let left = xs.length;
    xs.forEach((x, i) =>
      Promise.resolve(x).then(v => {
        out[i] = v;                // input order, not
finish order
        if (--left === 0) resolve(out);
      }, reject));                 // first rejection wins
  });
}
```

- Preserve input order, accept values/thenables, resolve an empty iterable immediately, reject on first rejection.

B. THE SIBLING COMBINATORS (ASKED AS FOLLOW-UPS)

```
// allSettled: wrap each side, never reject
const promiseAllSettled = xs => Promise.all(
  [...xs].map(x => Promise.resolve(x).then(
    value  => ({status: 'fulfilled', value}),
    reason => ({status: 'rejected',  reason}))));

// any: invert all — collect rejections,
// resolve on first fulfillment
function promiseAny(items) {
  const xs = [...items];
  return new Promise((resolve, reject) => {
    if (!xs.length)
      return reject(new AggregateError([], 'empty'));
    const errs = new Array(xs.length);
    let left = xs.length;
    xs.forEach((x, i) =>
      Promise.resolve(x).then(resolve, e => {
        errs[i] = e;
        if (--left === 0)
          reject(new AggregateError(errs));
      }));
  });
}
```

C. CONCURRENCY LIMITER

```
async function mapLimit(xs, limit, fn) {
  const out = new Array(xs.length);
  let next = 0;
  async function worker() {
    while (true) {
      const i = next++;            // safe: single-threaded
      if (i >= xs.length) return;
      out[i] = await fn(xs[i], i);
    }
  }
  await Promise.all(Array.from(
    {length: Math.min(limit, xs.length)}, worker));
  return out;
}
```

- Clarify fail-fast vs collect errors, cancellation, fairness, and whether synchronous throws count (here they become rejections via the async worker).

D. RETRY WITH BACKOFF + JITTER

```
async function retry(fn, {tries = 3, base = 200,
                          signal} = {}) {
  let err;
  for (let i = 0; i < tries; i++) {
    signal?.throwIfAborted();
    try { return await fn(); }
    catch (e) {
      err = e;
      if (!isTransient(e) || i === tries - 1) break;
      const delay = base * 2 ** i
                  + Math.random() * base;  // jitter
      await new Promise(r => setTimeout(r, delay));
    }
  }
  throw err;                       // keep original cause
}
```

- Narrate: transient-only, idempotency requirement, bounded attempts, abort between attempts, no retry stacking across layers.

> **TEST PATTERN**
> For every async utility: empty input, single item, all-reject, mixed settle order (use controlled resolvers, not timers), rapid repeated calls, and cancellation mid-flight.

09 Implementation prompts IV: data structures

A. EVENT EMITTER

```
class Emitter {
  #m = new Map();
  on(type, fn) {
    const s = this.#m.get(type) ?? new Set();
    s.add(fn); this.#m.set(type, s);
    return () => this.off(type, fn);  // unsubscribe
  }
  off(type, fn) { this.#m.get(type)?.delete(fn); }
  once(type, fn) {
    const un = this.on(type, (...a) => {
      un(); fn(...a);
    });
    return un;
  }
  emit(type, ...args) {
    for (const fn of [...(this.#m.get(type) ?? [])])
      fn(...args);      // snapshot: mutation-safe dispatch
  }
}
```

- Snapshot listeners before emit so add/remove during dispatch has defined behavior. Decide exception isolation (try/catch per listener?) aloud.

B. DEEP EQUAL

- **Contract first:** primitives/Object.is, arrays, plain objects, Date, Map/Set, prototypes, symbols, cycles?
- Recursive algorithm: identical → type/null guard → cycle-pair cache → tag-specific comparison → own-key sets → recurse values.
- Do not claim a 10-line plain-object solution handles every JS value. **Scope it honestly.**

```
function deepEqual(a, b, seen = new Map()) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object'
      || a === null || b === null) return false;
  if (seen.get(a) === b) return true;   // cycle pair
  seen.set(a, b);
  if (Array.isArray(a) !== Array.isArray(b)) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  if (ka.length !== kb.length) return false;
  return ka.every(k => Object.hasOwn(b, k)
      && deepEqual(a[k], b[k], seen));
}
// scoped: plain objects/arrays; extend for
// Date/Map/Set by tag before the key walk
```

C. FLATTEN / GET / GROUP

```
function flatten(a, out = []) {
  for (const x of a) Array.isArray(x)
    ? flatten(x, out) : out.push(x);
  return out;
}
// depth variant (Array.prototype.flat contract):
const flat = (a, d = 1) => d < 1 ? a.slice()
  : a.reduce((out, x) => out.concat(
      Array.isArray(x) ? flat(x, d - 1) : x), []);
```

```
const get = (o, path, def) => {
  const parts = Array.isArray(path) ? path
    : path.match(/[^.[\]]+/g) ?? [];
  let cur = o;
  for (const k of parts) {
    if (cur == null) return def;   // deliberate == null
    cur = cur[k];
  }
  return cur === undefined ? def : cur;
};

const groupBy = (xs, keyFn) => {
  const m = new Map();
  for (const x of xs) {
    const k = keyFn(x);
    (m.get(k) ?? m.set(k, []).get(k)).push(x);
  }
  return m;   // Object.groupBy exists but is newer
};
```

- Flatten: holes, depth, mutation, cycles. Get: path grammar, inherited properties, undefined vs missing, prototype-pollution safety if writing.

TEST PATTERN

For every utility: normal case, empty input, one item, duplicate calls, receiver/arguments, thrown error/rejection, mutation, and cleanup. Narrate the contract before optimizing.

10 JavaScript final drill

A. 60-SECOND LANGUAGE CHECK

QUESTION	ANSWER TO RETRIEVE
Why stale closure?	callback captured one render/ binding
Why lost this?	receiver determined by call site
Why A B C D?	sync job, microtasks, then task
Why sort wrong?	default string comparison
Why clone still mutates?	copy was shallow
Why fetch did not throw?	HTTP error is still a response
Why forEach finished early?	it ignores returned promises
Why Set missed duplicate object?	reference identity

B. CODE REVIEW CHECKLIST

- Inputs and contract stated; return type and mutation explicit.
- No accidental coercion; null/undefined and empty inputs handled.
- Receiver and closures correct; cleanup for timers/listeners/ requests.
- Promise rejection observed; concurrency and ordering intentional.
- Time/space complexity stated; hot path does not use shift/ splice accidentally.
- Tests include rapid repetition and failure, not only the happy path.

C. RUNTIME AVAILABILITY

- Conservative interview baseline: classic Map/Set/Array/ Promise APIs and AbortController.
- Feature-check newer conveniences: Set algebra, `Object.groupBy`, `AbortSignal.any/timeout`, iterator helpers, and the non-mutating array twins.
- Browser and Node event loops differ in phases and APIs. Explain the portable ordering guarantee the prompt needs rather than overclaiming internals.

D. PRIMARY REFERENCES

- **MDN JavaScript Guide and Reference:** developer.mozilla.org/docs/Web/JavaScript
- **MDN Web APIs** — DOM events, Fetch, AbortSignal, Streams, storage: developer.mozilla.org/docs/Web/API
- **ECMAScript language specification:** tc39.es/ecma262/
- **Node.js API documentation:** nodejs.org/docs/latest/api/

> **FINAL MOVE**
> When you do not remember an obscure API, write the small loop yourself. Interviewers reward a correct mental model and explicit edge cases more than clever syntax.